

Student Handbook

Docker Training

Day 3 · Session 1

Docker Compose fundamentals

Learning objectives

By the end of this session you will be able to:

- Explain the structure of a docker-compose.yml file and map its keys to docker run flags
- Write a Compose file with multiple services, a shared network, and a named volume
- Add a healthcheck to a service and use condition: service_healthy in depends_on
- Use a .env file for configuration and reference variables with \${VARIABLE} syntax
- Use profiles to define optional services that only start on demand
- Use docker compose config to validate and preview the resolved file
- Use the core Compose workflow: up, down, logs, exec, run, ps, restart
- Understand the difference between docker compose down and docker compose down -v
- Create a docker-compose.override.yml for environment-specific configuration

Compose vs manual docker run

Everything you did manually in Day 1 Session 3 is replaced by a single command. Here is the comparison:

Manual (Day 1 approach)	Compose (Day 2 approach)
<pre>docker network create appnet docker volume create pgdata docker run -d --name db --network appnet -e POSTGRES_PASSWORD=... -v pgdata:/var/lib/postgresql/data postgres:16 docker run -d --name api --network appnet -p 8080:8080 -e ConnectionStrings__... weatherapi:1.0 docker run -d --name ui --network appnet -p 4200:80 weather-ui:1.0</pre>	<pre>docker compose up -d</pre> That's it. One command.

Key concepts

services: Root section. Each named block is one container. Keys map 1-to-1 with docker run flags.	networks: Declared at root level. Compose auto-creates user-defined bridges with DNS. Services join by name.
volumes: Declared at root level. Compose creates named volumes automatically. Mount per-service with	depends_on Controls startup order. With condition: service_healthy it waits for a healthcheck to

volumes: key.	pass.
healthcheck A command Docker polls periodically. Until it passes, dependent services with service_healthy wait.	.env file KEY=VALUE pairs auto-loaded by Compose. Reference as \${KEY} anywhere in the compose file.
profiles: Tag services with profiles: [name]. Only started when --profile name is passed to docker compose up.	env_file: Load env vars from a file into a specific service container. Different from the root .env file.

Compose file anatomy — annotated

Read through this complete docker-compose.yml carefully. Every line is explained in a comment above it.

```
# docker-compose.yml
# Declares all services, networks and volumes for this project.
# Compose reads this file when you run docker compose up.

services:

  # — PostgreSQL —————
  db:
    image: postgres:16
    restart: unless-stopped
    compose stop
    environment:
      POSTGRES_USER: ${DB_USER}
      POSTGRES_PASSWORD: ${DB_PASS}
      POSTGRES_DB: ${DB_NAME}
    volumes:
      - pgdata:/var/lib/postgresql/data
    compose down
    networks:
      - appnet
    appnet
    healthcheck:
      test: ["CMD", "pg_isready", "-U", "${DB_USER}"]
    when ready
      interval: 5s
      timeout: 5s
      retries: 5
    failures
      start_period: 10s
    count)

  # — pgAdmin (optional tool — only with --profile tools) —————
  pgadmin:
    image: dpage/pgadmin4:latest
    depends_on:
      db:
```

```

        condition: service_healthy      # Wait for db healthcheck to pass before
starting
    environment:
        PGADMIN_DEFAULT_EMAIL: ${PGADMIN_EMAIL}
        PGADMIN_DEFAULT_PASSWORD: ${PGADMIN_PASS}
    ports:
        - "${PGADMIN_PORT:-5050}:80"    # :-5050 = default value if PGADMIN_PORT not
set
    networks:
        - appnet
    profiles:
        - tools                          # Only starts when: docker compose --profile
tools up

# — Top-level volume declarations —————
volumes:
    pgdata:                             # Compose creates this — leave body empty
for defaults

# — Top-level network declarations —————
networks:
    appnet:                             # User-defined bridge with embedded DNS

```

Hands-on lab steps

1

Clean up the Day 1 environment

Start fresh before Compose takes over network and volume management

Remove Day 1 containers and manually created networks

```

# Stop and remove Day 1 containers (errors ignored if already gone)
docker rm -f db api ui ui-dev 2>/dev/null || true

# Remove the manually created network from Day 1
docker network rm appnet 2>/dev/null || true

# Verify only default networks remain
docker network ls
docker ps -a

```

Expected output:

NETWORK ID	NAME	DRIVER	SCOPE
...	bridge	bridge	local
...	host	host	local
...	none	null	local

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
# (empty — no containers running)						

Why clean up first

Compose will create its own prefixed network (compose-lab_appnet) and volume (compose-lab_pgdata). If old resources with the same name exist from Day 1, Compose may reuse them or fail with a conflict. Starting clean avoids confusion.

2

Create project folder and .env file

Configuration via .env — never hardcode secrets in the compose file

Create the project structure

```
mkdir compose-lab && cd compose-lab
touch .env
touch docker-compose.yml
ls -la
```

Expected output:

```
-rw-r--r--  1 user user    0 Jun 17 09:10 .env
-rw-r--r--  1 user user    0 Jun 17 09:10 docker-compose.yml
```

.env — paste this content (open in any text editor)

```
# .env — loaded automatically by docker compose
# NEVER commit this file with real credentials to git

# PostgreSQL
DB_USER=postgres
DB_PASS=Training123
DB_NAME=traindb

# pgAdmin
PGADMIN_EMAIL=admin@training.com
PGADMIN_PASS=admin123
PGADMIN_PORT=5050
```

Create .gitignore and .env.example

```
# Add .env to gitignore — real credentials stay local
echo ".env" >> .gitignore

# .env.example has keys but no values — safe to commit
cat > .env.example << 'EOF'
DB_USER=
DB_PASS=
DB_NAME=
PGADMIN_EMAIL=
PGADMIN_PASS=
PGADMIN_PORT=5050
EOF
```

Never commit .env

The .env file must always be in .gitignore. Commit only .env.example with empty values so teammates know which variables are required. Anyone who clones the repo copies .env.example to .env and fills in their own values.

Variable substitution syntax

`${DB_PASS}` reads the value from .env. `${PGADMIN_PORT:-5050}` reads from .env but falls back to 5050 if the variable is not set — the `:-` syntax provides a default value.

3

Write the Compose file — PostgreSQL with healthcheck

One service first; validate it before adding dependencies

docker-compose.yml — paste this complete content

```
services:

  db:
    image: postgres:16
    restart: unless-stopped
    environment:
      POSTGRES_USER: ${DB_USER}
      POSTGRES_PASSWORD: ${DB_PASS}
      POSTGRES_DB: ${DB_NAME}
    volumes:
      - pgdata:/var/lib/postgresql/data
    networks:
      - appnet
    healthcheck:
      test: ["CMD", "pg_isready", "-U", "${DB_USER}"]
      interval: 5s
      timeout: 5s
      retries: 5
      start_period: 10s

volumes:
  pgdata:

networks:
  appnet:
```

Validate syntax and preview resolved variables

```
# Shows the fully resolved file with all ${} substitutions applied
# Any undefined variables or syntax errors appear here
docker compose config
```

Expected output:

```
name: compose-lab
services:
  db:
    environment:
      POSTGRES_DB: traindb
      POSTGRES_PASSWORD: Training123
      POSTGRES_USER: postgres
    healthcheck:
      test:
        - CMD
        - pg_isready
        - -U
        - postgres
    image: postgres:16
    restart: unless-stopped
    ...
```

Start just the database service

```
docker compose up -d db
```

Expected output:

```
[+] Running 3/3
  Network compose-lab_appnet   Created
  Volume "compose-lab_pgdata"  Created
  Container compose-lab-db-1    Started
```

Wait for the healthcheck to pass

```
# Check status — wait until STATUS shows (healthy)
docker compose ps
```

Expected output:

NAME	IMAGE	STATUS
compose-lab-db-1	postgres:16	Up 12 seconds (healthy)

Compose auto-prefixes resource names

Compose prefixes all resources with the project name (the folder name, compose-lab). The network becomes compose-lab_appnet and the volume becomes compose-lab_pgdata. This prevents name collisions between different Compose projects on the same machine.

Healthcheck status progression

A newly started container goes through: starting (grace period) → healthy (pg_isready returns 0) or unhealthy (five consecutive failures). Dependent services with condition: service_healthy will not start until the status reaches healthy.

4

Add pgAdmin with depends_on and profiles

Dependency ordering and optional services via profiles

Add the pgAdmin service — append inside services: in your compose file

```
# Append this block inside services: after the db service
pgadmin:
  image: dpage/pgadmin4:latest
  depends_on:
    db:
      condition: service_healthy
  environment:
    PGADMIN_DEFAULT_EMAIL: ${PGADMIN_EMAIL}
    PGADMIN_DEFAULT_PASSWORD: ${PGADMIN_PASS}
  ports:
    - "${PGADMIN_PORT:-5050}:80"
  networks:
    - appnet
  profiles:
    - tools
```

Start WITHOUT the profile — pgAdmin is excluded

```
docker compose up -d
```

Expected output:

```
[+] Running 1/1
  Container compose-lab-db-1    Running
```

```
# pgAdmin is NOT started — it has a profile assigned
```

Start WITH the tools profile — pgAdmin is included

```
docker compose --profile tools up -d
```

Expected output:

```
[+] Running 2/2
   Container compose-lab-db-1      Running
   Container compose-lab-pgadmin-1 Started

# pgAdmin waited for db to become healthy before starting
```

Connect pgAdmin to the database

```
# Open http://localhost:5050 in your browser
# Email:      admin@training.com
# Password:  admin123

# In pgAdmin UI: right-click Servers > Register > Server
# Tab General:      Name: traindb
# Tab Connection:  Host: db              <- container name, DNS resolves it
#                  Port: 5432
#                  Username: postgres
#                  Password: Training123
```

condition: service_healthy vs condition: service_started

`service_started` (the default) only waits for the container process to start — not for the application inside to be ready. `service_healthy` waits for the healthcheck to pass. Always use `service_healthy` for databases, because postgres takes a few seconds after process start before it accepts connections.

Profiles keep your compose file clean

Profiles let you define dev tools (pgAdmin, Redis Commander, MailHog) in the same file without polluting the default startup. CI/CD pipelines run `docker compose up -d` with no profile. Developers add `--profile tools` when they need the UI tools.

5 Master the daily Compose workflow

Logs, exec, run, restart, and the down vs down -v distinction

Tail logs — all services, then a single service

```
# All services — Ctrl+C to stop following
docker compose logs -f

# Single service only
docker compose logs -f db

# Last 50 lines then follow
docker compose logs --tail=50 -f db
```

Exec into a running service container


```
# Interactive psql session inside the db service
docker compose exec db psql -U postgres traindb
```

```
# Non-interactive — run a command and return
docker compose exec db pg_isready -U postgres
```

Expected output:

```
/var/run/postgresql:5432 - accepting connections
```

Run a one-off command in a fresh container

```
# docker compose run starts a NEW container (does not affect running ones)
# --rm removes the container when the command finishes
docker compose run --rm db psql -U postgres traindb -c "\\1"
```

Expected output:

List of databases			
Name	Owner	Encoding	...
traindb	postgres	UTF8	...
postgres	postgres	UTF8	...

Restart a single service without touching others

```
docker compose restart db
```

Understand down vs down -v — the most important distinction

```
# Stop all services — containers removed, volumes and network KEPT
docker compose down
docker volume ls
# compose-lab_pgdata is still listed — data is safe

# Start again — data is still there
docker compose up -d
docker compose exec db psql -U postgres traindb -c "SELECT datname FROM
pg_database;"

# Stop AND delete all named volumes — DATA IS PERMANENTLY DELETED
docker compose down -v
docker volume ls
# compose-lab_pgdata is gone
```

docker compose down -v in production

Never run `docker compose down -v` on a production database. It deletes all named volumes — all your data — immediately and permanently. Reserve `-v` for CI/CD clean builds and local resets only. In production, remove containers with `docker compose down` and leave volumes intact.

6

Use override files for dev vs prod configuration

`docker-compose.override.yml` is auto-merged — the standard multi-environment pattern

Compose automatically merges `docker-compose.yml` and `docker-compose.override.yml` when both exist in the same directory. The override file is additive — it extends rather than replaces.

Create docker-compose.override.yml — dev-only additions

```
# docker-compose.override.yml
# Auto-merged with docker-compose.yml on every docker compose up
# Contains dev-only settings — do not use in production

services:
  db:
    ports:
      - "5432:5432"      # Expose DB port so DBeaver can connect directly
                        # Remove this in production for security

  pgadmin:
    environment:
      PGADMIN_CONFIG_SERVER_MODE: "False"  # Skip login screen in dev
```

Verify Compose merges both files automatically

```
# docker compose config shows the MERGED result
docker compose config | grep -A 5 "ports:"
```

Expected output:

```
ports:
  - mode: ingress
    target: 5432
    published: "5432"
    protocol: tcp
# The port mapping from override.yml appears in the merged output
```

Explicitly select files for production (skip override)

```
# Use only the base file — override.yml is ignored
docker compose -f docker-compose.yml up -d

# Combine base + a separate prod file
docker compose -f docker-compose.yml -f docker-compose.prod.yml up -d
```

Final check — confirm services are healthy

```
docker compose ps
```

Expected output:

NAME	IMAGE	STATUS
compose-lab-db-1	postgres:16	Up 2 minutes (healthy)
compose-lab-pgadmin-1	dpage/pgadmin4	Up 45 seconds

Session outcome

You have a fully declarative Compose setup with: PostgreSQL with a health check, pgAdmin behind a profile, .env-based configuration, healthcheck-ordered dependencies, and an override file for dev extras. This is the foundation the full stack (Session 2) builds on.

Quick reference — Compose commands

Command	What it does
<code>docker compose up -d</code>	Start all services in background (creates network + volumes)
<code>docker compose up -d --build</code>	Rebuild images then start
<code>docker compose up -d db api</code>	Start specific services only
<code>docker compose --profile tools up -d</code>	Start default services + those tagged with tools profile
<code>docker compose down</code>	Stop and remove containers (volumes and network kept)
<code>docker compose down -v</code>	Stop, remove containers AND delete named volumes
<code>docker compose down --rmi all</code>	Stop, remove containers and all images
<code>docker compose ps</code>	List service containers and their status
<code>docker compose logs -f</code>	Tail logs for all services
<code>docker compose logs -f api</code>	Tail logs for one service
<code>docker compose exec db bash</code>	Open shell in a running service container
<code>docker compose run --rm db cmd</code>	Run one-off command in a new container, then remove it
<code>docker compose build</code>	Build all images without starting
<code>docker compose build api</code>	Build a single service image
<code>docker compose config</code>	Show merged, resolved compose file (validates syntax + variables)
<code>docker compose restart api</code>	Restart a single service
<code>docker compose pull</code>	Pull latest images for all services
<code>docker compose -f base.yml up -d</code>	Use a specific compose file, skipping override.yml

Compose file key reference

Key	Purpose and notes
<code>image:</code>	Docker image to use. Cannot be used with <code>build:</code> in the same service.
<code>build:</code>	Build context (path to Dockerfile). Used instead of <code>image:</code> for custom images.
<code>container_name:</code>	Override the auto-generated container name. Loses auto-scaling.

<code>restart:</code>	unless-stopped: restart on crash, stop on compose stop. always: restart always.
<code>environment:</code>	Key-value env vars. Use <code>\${VAR}</code> to read from <code>.env</code> file.
<code>env_file:</code>	Load env vars from a file into the container. Separate from root <code>.env</code> .
<code>ports:</code>	HOST:CONTAINER port mapping. Quoted strings avoid YAML parsing issues.
<code>volumes:</code>	Named volumes (name:/path) or bind mounts (./host:/container).
<code>networks:</code>	Networks this service joins. Service name is DNS hostname on each network.
<code>depends_on:</code>	Start order. Add condition: <code>service_healthy</code> to wait for healthcheck.
<code>healthcheck:</code>	test, interval, timeout, retries, start_period. Sets container health status.
<code>profiles:</code>	List of profile names. Service only starts when <code>--profile</code> name is passed.
<code>command:</code>	Override the default CMD from the image.
<code>entrypoint:</code>	Override the default ENTRYPOINT from the image.
<code>working_dir:</code>	Override WORKDIR from the image.
<code>user:</code>	Override USER from the image.

Common mistakes to avoid

Mistake 1 — Using `depends_on` without a healthcheck condition

`depends_on`: db by itself only waits for the db container process to start — not for PostgreSQL to be ready to accept connections. The API will start and immediately fail to connect. Always pair `depends_on` with condition: `service_healthy` and add a healthcheck to the dependency.

Mistake 2 — Hardcoding passwords in the compose file

Writing `POSTGRES_PASSWORD: Training123` directly in `docker-compose.yml` means your password is committed to git. Always use `${DB_PASS}` and store the actual value in `.env`. The `.env` file must be in `.gitignore`.

Mistake 3 — Running `docker compose down -v` to 'clean up'

`docker compose down -v` deletes all named volumes, which means all database data. Use plain `docker compose down` to stop services while keeping data. Only use `-v` when you genuinely want to wipe the database (CI/CD clean builds, or resetting a dev environment).

Mistake 4 — Forgetting to declare volumes and networks at the top level

A volume or network used inside a service must also be declared at the root level (volumes: `pgdata`; and networks: `appnet`). If you only declare them inside a service, Compose raises a validation error.

Leave the body empty for default settings.

Mistake 5 — YAML indentation errors

YAML is whitespace-sensitive. Two spaces per indent level — never tabs. A misaligned key silently becomes a string value or raises a parse error. Run `docker compose config` after every edit to catch issues before running up.

Coming up — Session 2: Full stack with Compose

In Session 2 you will add the .NET API and Angular UI to the Compose file so the entire application starts with a single command.

- Add the api service using `build:` to compile from the Dockerfile
- Add the ui service and configure CORS between Angular and the .NET API
- Wire all three services on appnet so the API connects to PostgreSQL by service name
- Add a `docker-compose.override.yml` for Angular hot-reload with bind mounts
- Perform a hot-swap: rebuild and replace a single service without stopping the others

Keep your compose-lab folder

The `docker-compose.yml` and `.env` you created today will be extended in Session 2. Leave the db container running — the next session builds directly on top of this setup.